

Puppet Debugger Presentation

performed by [Roberto Nogueira](#)

Objective



Puppet Debugger

It is to introduce a [Puppet Debugger](#), a command line tool to help you write, understand, and debug puppet code. In order to achieve that the Puppet and its language is revised.

PuppetDebugger



A command line tool to help you write, understand, and debug puppet code.



[Demo](#) is an Online Puppet Debugger.

Puppet



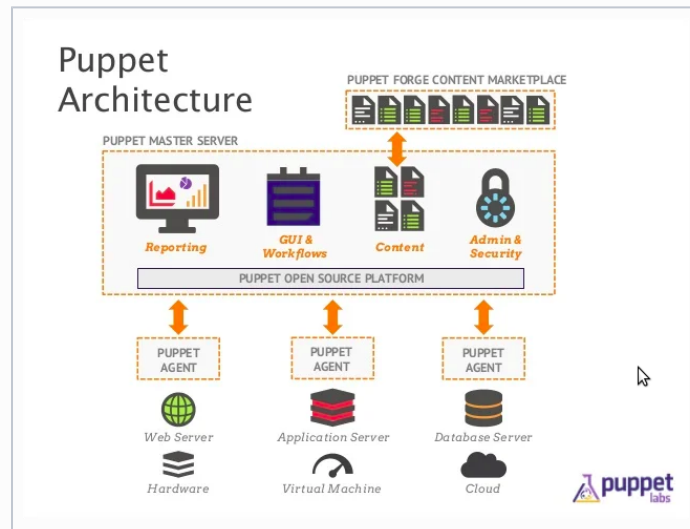
Puppet

Puppet is a [software configuration management](#) tool which includes its own [declarative language](#) to describe [system configuration](#). It is a [model-driven solution](#) that requires limited programming knowledge to use.^[3]

Puppet Architecture

Puppet usually follows [client-server](#) architecture. The client is known as an agent and the server is known as the master. For testing and simple configuration, it can also be used as a stand-alone application run from the command line.

Puppet Server is installed on one or more servers, and Puppet Agent is installed on all the machines to be managed. Puppet Agents communicate with the server and fetch configuration instructions. The Agent then applies the configuration on the system and sends a status report to the server.^[11] Devices can run Puppet Agent as a [daemon](#), that can be triggered periodically as a [cron](#) job or can be run manually whenever needed.



Tip

In order to install **Puppet** the way it is described here [Docker](#) is required see [Ref.1](#) for further installation.

Install Puppet and Puppet Debugger in Debian

```
docker run --name=debian11 -t -i --rm debian:11 bash

apt update
apt install systemctl wget lsb-release -y
wget https://apt.puppetlabs.com/puppet7-release-bionic.deb
dpkg -i puppet7-release-bionic.deb
apt update
apt-get install puppet-agent -y
echo 'export PATH=$PATH:/opt/puppetlabs/puppet/bin' >> ~/.bashrc
source ~/.bashrc
gem install --no-document puppet-debugger
```



Puppet relevant commands

```
# print puppet configuration, usually has to be execute with sudo
puppet config print
# set log level to debug, usually has to be execute with sudo
puppet config set log_level debug
```

Puppet Language Basics Introduction



Introduction

Puppet consists of a custom **declarative language** to describe system configuration, which can be either applied directly on the system, or compiled into a catalog and distributed to the target system via **client-server paradigm** (using a **REST API**), and the agent uses system specific **providers** to enforce the resource specified in the manifests. The resource abstraction layer enables administrators to describe the configuration in high-level terms, such as users, services and packages without the need to specify OS specific commands (such as **rpm**, **yum**, **apt**).

Puppet is **model-driven**, requiring limited programming knowledge to use.^[10]

DESCRIPTION



IMPLEMENTATION



Tip

See also in [Introduction of Puppet Language Basics](#) in Puppet channel in YouTube and homepage.

Resources



Resources

Are the fundamental **building blocks** of the Puppet Language.

Is characteristic of a **node** that needs to be managed overtime.



Resource Declaration adds a resource to the catalog and tells Puppet to manage that resource state.

Resource Declaration	Example
<pre>resource_type { 'name': attribute => value, }</pre>	<pre>file { '/etc/motd': ensure => 'file' content => 'Hell owner => 'root' group => 'root' mode => '0644' }</pre>

Resources Types are e.g. User, Package and Service.

Examples in Debian

Check the existent resources types

```
puppet resource --types
:
```

- augeas
- cron
- exec
- file
- filebucket
- group
- host
- mount
- notify
- package

```
:
```

Check the existent resource file type

```
puppet resource file '/etc/hosts'
file { '/etc/hosts':
  ensure => 'file',
  content => '{sha256}674a2d0859a6ed81ce1e3513f6ee1f710b47c3b97301b284124d5b95cf566b47',
  ctime   => '2022-05-21 12:17:40 +0000',
  group   => 0,
  mode    => '0644',
  mtime   => '2022-05-21 12:17:40 +0000',
  owner    => 0,
  provider => 'posix',
  type     => 'file',
}
```

Check the existent resource service type

```
service --status-all
[ ? ] hwclock.sh
[ - ] procps
```

```
puppet resource service 'procps'
```

```
service { 'procps':
  ensure => 'running',
  enable => 'false',
  provider => 'debian',
}
```

Installing a resource package type

```
puppet resource package 'tree' ensure=present
```

Notice: /Package[tree]/ensure: created

```
package { 'tree':
  ensure => 'purged',
  provider => 'apt',
}
```



Tip

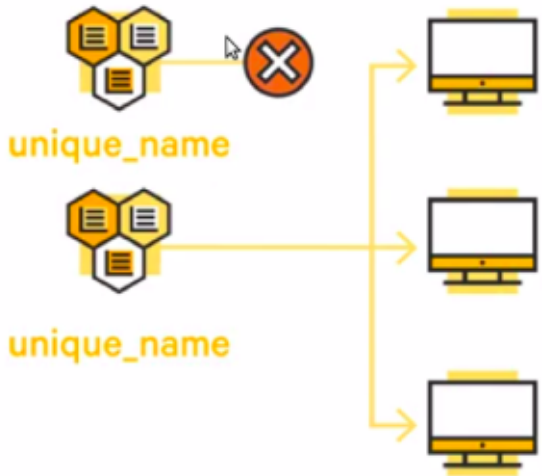
See also in [Resources](#) of [Puppet Language Basics](#) in Puppet channel in YouTube and homepage.

Classes



Classes

Multiple Puppet Resources can be collected and distributed together using **Classes**, by combining Resources into a Class. A Class can configure a [large block of functionality](#).



```
class ssh {  
  package { 'openssh-clients':  
    ensure => present,  
  }  
  file { 'etc/ssh/ssh_config':  
    ensure => file,  
    content => template('ssh/ssh_config'  
  }  
  service { 'sshd':  
    ensure => running,  
    enable => true,  
  }  
}
```



Tip

See also in [Classes](#) of [Puppet Language Basics](#) in Puppet channel in YouTube and homepage.

Modules



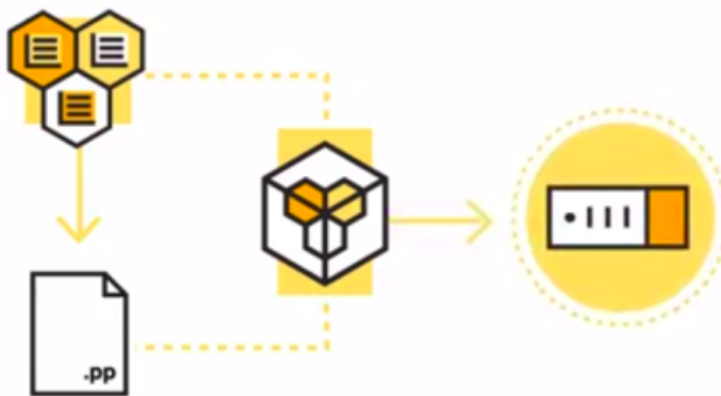
Modules

Are one of the most basic building blocks of the Puppet Language. They are used to manage a specific task or an application such as installing or configuring a piece of software.

Module is simply a directory tree that contains the files needed for a given configuration. This directory structure allows nodes to discover and load Classes, Resources Types and more.

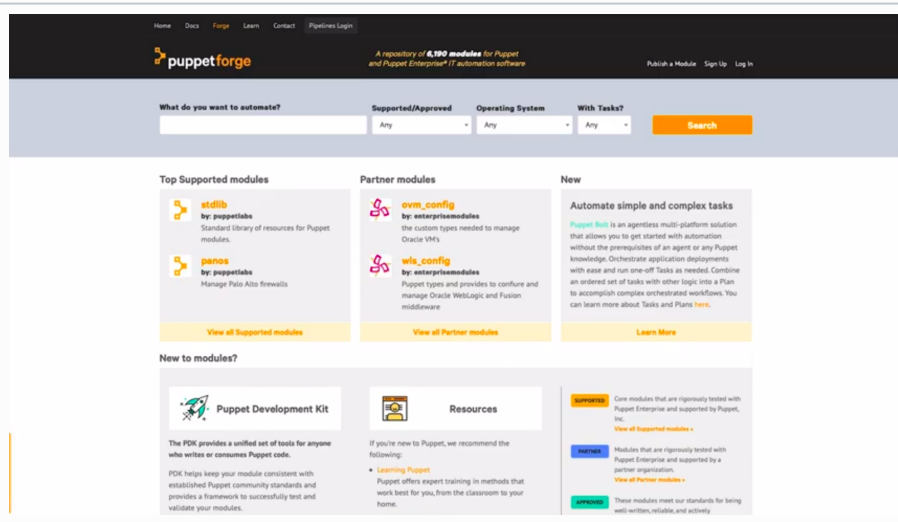
module-name is the name of the top's directory tree.

If a Class is defined in a Module you can declare that name in any Manifest. When Puppet Master compiles the catalog it will dynamically load the Manifest that contain the Class definition.



```
module-r
|- manif
|- ini
|- files
|- templ
|- exampl
|- spec
|- metac
```

There are thousands of Modules available for downloading at [Puppet Forge](https://puppetforge.com), written by Puppet developers and community members. There are compatible with multi-platforms.



Puppet Module relevant commands

```
# list puppet modules and its path, usually has to be execute with sudo
puppet module list
puppet config print modulepath
# install puppet modules, usually has to be execute with sudo
puppet module install stdlib
```



Tip

See also in [Modules of Puppet Language Basics](#) in Puppet channel in YouTube and homepage.

Variables



Variables

Exists in order to be interpolated in strings to avoid rework in case the value is updated.

```
file { 'path/to/file/share/directory1':  
  ensure => directory,  
}  
  
file { 'path/to/file/share/directory2':  
  ensure => directory,  
}  
  
file { 'path/to/file/share/directory3':  
  ensure => directory,  
}
```

```
$file_share_path = 'new/p  
  
file { "${file_share_path  
  ensure => directory,  
}  
  
file { "${file_share_path  
  ensure => directory,  
}  
  
file { "${file_share_path  
  ensure => directory,  
}
```



Tip

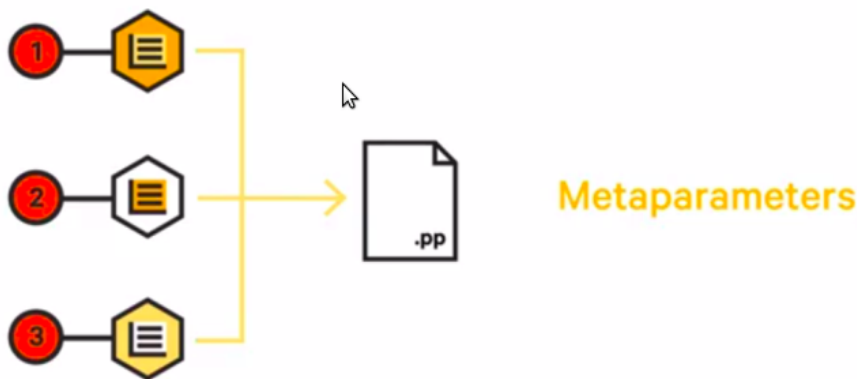
See also in [Variables](#) of [Puppet Language Basics](#) in Puppet channel in YouTube and homepage.

Metaparameters



Metaparameters

When a Puppet manifest is loaded the resources inside the manifest are managed the order they are declared. But when you need a group of resources to be managed in different or specific order you must explicit declare the relationships there where the metaparameters come in. Metaparameters are special attributes that can be used in any resource type, and specify how your system act to the resource rather than mapping directly to the system state.



The Puppet Language uses four Metaparameters to establish relationships.

- **'before'** causes a resource to be applied before the target resource.
- **'require'** causes a resource to be applied after the target resource.
- **'notify'** causes a resource to be applied *before* the target resource, but will refresh only if the notifying resource changes.
- **'subscribe'** causes a resource to be applied *after* the target resource, again refreshing only if the target resource changes.

```
package { 'openssh'
  ensure => present
  before => File
}
```

```
file { '/etc/ssh/sshd_config':
  ensure => file,
  mode   => 600,
  source => 'puppet:///modules/sshd/sshd_config',
  require => Package['openssh-server'],
}
```

```
service { 'sshd':
  ensure   => running
  enable   => true
  subscribe => File
}
```



Tip

See also in [Metaparameters](#) of [Puppet Language Basics](#) in Puppet channel in YouTube and homepage.

Puppet Debugger



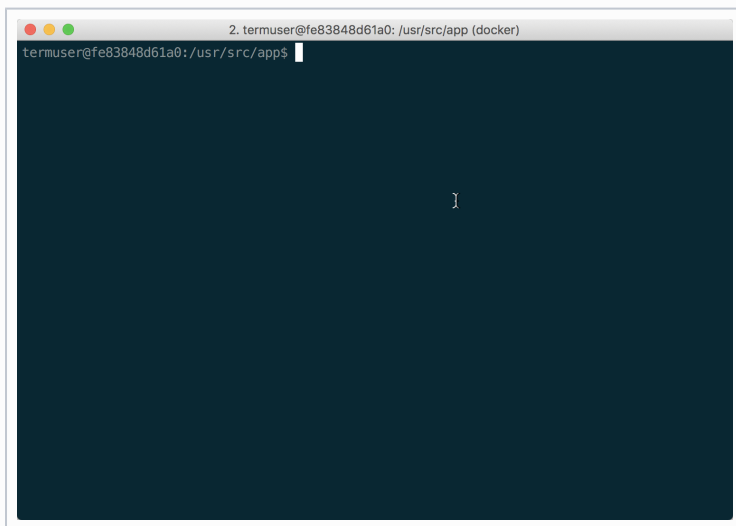
Puppet Debugger

The debugger is a puppet application packaged as a ruby gem installed along side puppet. The primary purpose is to give you an interactive console to debug puppet code and allow you tinker with the puppet language.

Since the beginning of puppet there has never been a simple way to run arbitrary puppet code without affecting a real system. We have always used simple variable dump hacks like puppet apply and a notify resource to spit out the result of some code. The debugger replaces that entire puppet apply hack with a simple interactive console.

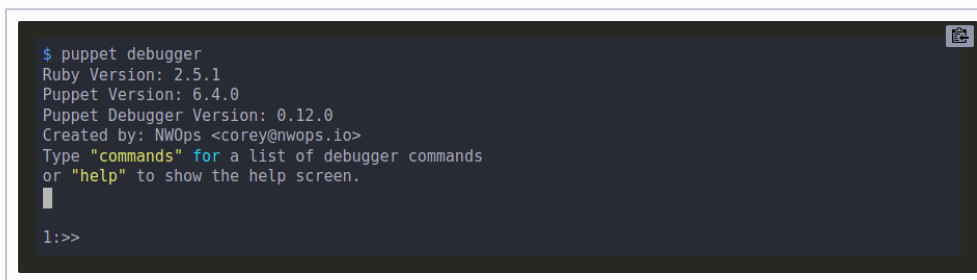
Almost all programming languages provide a debugging console that let you play around with code interactively, except Puppet. Thus the debugger was born.

The debugger parses and evaluates your code and shows you the resultant value instantly. Immediate feedback helps you learn faster, gain insight into how the language or your code will actually work.



BASIC USAGE

Once you have the debugger started, use the commands command to show a list of commands.



At any point you can type any valid puppet code. Start with a few simple things then move onto more advanced puppet concepts. Just remember "garbage in, garbage out". Syntax errors, duplicate resources, and variable re-definitions will show their relevant errors.

1. Variable assignment: `$var1 = 'kaboom!'`
2. Run a function: `sha1('12345')`
3. include a class `include stdlib`



COMMANDS

The debugger comes with many commands that allow you to inspect the catalog, your code or reference items currently in scope. All commands are plugins that can process, modify or perform some action on the input and/or output of the debugger.

The menu on the left details what each command does. See [Puppet Debugger Commands](#).

Resources

RESOURCES

Summary

List all the resources currently in the catalog. Remember the puppet debugger is like an interactive manifest file. When you type puppet code, puppet will evaluate it and add it to the catalog.

```
1:>> include nodejs
2:>> resources
Resources not shown in any specific order
[
  [ 0] "Stage['main']",
  [ 1] "Class['Settings']",
  [ 2] "Class['main']",
  [ 3] "Node['_node_regexp__foo']",
  [ 4] "Notify['found foo node']",
  [ 5] "Class['Nodejs::Params']",
  [ 6] "Class['Nodejs']",
  [ 7] "Class['Nodejs::Install']",
  [ 8] "Package['nodejs']",
  [ 9] "Package['nodejs-devel']",
  [10] "Package['nodejs-debuginfo']",
  [11] "Package['npm']",
  [12] "File['root_npmrc']",
  [13] "Class['Nodejs::Repo::Nodesource']",
  [14] "Class['Nodejs::Repo::Nodesource::Yum']",
  [15] "Yumrepo['nodesource']",
  [16] "Yumrepo['nodesource-source']",
  [17] "File['/etc/pki/rpm-gpg/NODESOURCE-GPG-SIGNING-KEY-EL']"
]
```



For a more complete reference about [Resources](#) lookup in the Puppet Debugger homepage.

Variables

VARIABLES

Variables in puppet code help us remove statically defined data. Unfortunately they also have the side affect of being misinterpreted due to logic bugs or unforeseen circumstances.

You can use the puppet debugger to see the values of all variables no matter which scope they are defined in. This allows you to see problems ahead of time or even help validate your logic.

```
8:>> $file_base_path = "/tmp"
=> "/tmp"
9:>> $file_path = "${file_base_path}/test.txt"
=> "/tmp/test.txt"
10:>>
```

```
10:>> ls
"Facts were removed for easier viewing"
{
  "datacenter" => "datacenter1",
  "facts"      => "removed by the puppet-debugger",
  "file_base_path" => "/tmp",
  "file_path"    => "/tmp/test.txt",
  "module_name"  => "",
  "name"         => "main",
  "server_facts" => {
    "environment" => "production",
    "serverip"    => nil,
    "servername"  => "MacBook-Pro-162.local",
    "serverversion" => "6.4.0"
  },
  "title"       => "main",
  "trusted"     => {
    "authenticated" => "local",
    "certname"      => nil,
    "domain"        => nil,
    "extensions"    => {},
    "hostname"      => nil
  }
}
```

✓ For a more complete reference about [Variables](#) lookup in the Puppet Debugger homepage.

Benchmark

BENCHMARK

Did you know you can benchmark your puppet code? The benchmark command will time how long it takes to compile your code.

Discover which functions are slowing you down by timing them with benchmark.

The benchmark command is a toggle switch which enables/disables benchmarking. A shortcut is also provided by typing the `bm` command.

```
1>> benchmark
Benchmark Mode On
2:BM>> sha256('fdsafdfasdfsdfasdfsdfafdsfsdfasdfsdfasdfsdfasdfsdf')
=> [
  [0] "d2fc2cee2205703df0245bcadceb5677703bd7ba8742a82b77683c7d45d11ff0",
  [1] "Time elapsed 6.99 ms"
]
3:BM>> sha256('fdsafdfasdfsdfasdfsdfafdsfsdfasdfsdfasdfsdfasdfsdf')
=> [
  [0] "d2fc2cee2205703df0245bcadceb5677703bd7ba8742a82b77683c7d45d11ff0",
  [1] "Time elapsed 0.47 ms"
]
4:BM>> benchmark
Benchmark Mode Off
```

✓ For a more complete reference about [Benchmark](#) lookup in the Puppet Debugger homepage.

Ls

LS

When writing puppet code there will be times when you need to see the value of a variable. Inside the debugger you can just type the variable name. The `ls` command will list all the variables currently set, in the current scope and even in the scope of a evaluated resource.

The scope used during Puppet compilation can be inspected at any time with the `ls` command.

Use the `vars` or `ls` command to show the contents of the current scope.

```
8:>> ls
"Facts were removed for easier viewing"
{
  "datacenter" => "datacenter1",
  "facts"      => "removed by the puppet-debugger",
  "module_name" => "",
  "name"       => "main",
  "server_facts" => {
    "environment" => "production",
    "serverip"    => "10.0.3.154",
    "servername"  => "MacBook-Pro-162",
    "serverversion" => "6.4.0"
  },
  "title"      => "main",
  "trusted"    => {
    "authenticated" => "local",
    "certname"      => nil,
    "domain"        => nil,
    "extensions"    => {},
    "hostname"      => nil
  }
}
```

✓ For a more complete reference about [LS](#) lookup in the Puppet Debugger homepage.

Set

SET

The set command is used to set various puppet debugger settings. The loglevel is probably the most useful.

Currently the only settings are loglevel and node. You can use autocomplete (tab) to help you find the options for the setting.

```
11:>> set loglevel debug
loglevel debug is set
```



For a more complete reference about [Set](#) lookup in the Puppet Debugger homepage.

Krt

KNOWN RESOURCE TYPES

List all of the known resource types with the command krt. This shows all the resource types in the current catalog.

```
5:>> krt
{
  :applications      => [],
  :capability_mappings => [],
  :definitions       => [],
  :hostclasses       => [
    [0] "",
    [1] "settings",
    [2] "stdlib",
    [3] "stdlib:stages"
  ],
  :nodes             => [],
  :sites              => nil
}
```



For a more complete reference about [Krt](#) lookup in the Puppet Debugger homepage.

Classes

CLASSES

The classes command list all the class level resources found in the current catalog. A new catalog is compiled with each entry so the list will grow over as you use more classes.

```
4:>> include stdlib
=> Puppet::Type::Component {
  loglevel => notice,
  name => "Stdlib",
  title => "Class[Stdlib]"
}
5:>> classes
[
  [0] "settings",
  [1] "stdlib",
  [2] "stdlib:stages"
]
5:>>
```



For a more complete reference about [Classes](#) lookup in the Puppet Debugger homepage.

Facts

FACTS

The debugger will get facts from FacterDB to speed up the initial loading process. These facts are then used in the scope during compilation.

To list the facts in the scope simply run the facts command or \$: : facts

```
6:>> facts
{
  "architecture"      => "x86_64",
  "augeas"             => {
    "version" => "1.4.0"
  },
  "augeasversion"      => "1.4.0",
  "bios_release_date"  => "12/01/2006",
  "bios_vendor"        => "Innotek GmbH",
}

6:>> $facts
{
  "architecture"      => "x86_64",
  "augeas"             => {
    "version" => "1.4.0"
  },
  "augeasversion"      => "1.4.0",
  "bios_release_date"  => "12/01/2006",
  "bios_vendor"        => "Innotek GmbH",
}
```

You can force the debugger to use real facts from the current node by disabling FactorDB:

```
puppet debugger --no-facterdb --test -e '$os[family]'
```

Reference any fact

You can reference any fact just like you would inside a manifest (because the debugger is an interactive manifest). This is handy if you need to know the data structure of a fact before writing code against it.

```
1:>> $os
=> {
  "architecture" => "x86_64",
  "family"       => "RedHat",
  "hardware"     => "x86_64",
  "name"         => "Fedora",
  "release"      => {
    "full" => "23",
    "major" => "23"
  },
  "selinux"      => {
    "config_mode" => "permissive",
    "config_policy" => "targeted",
    "current_mode" => "permissive",
    "enabled" => true,
    "enforced" => false,
    "policy_version" => "29"
  }
}
2:>> $os[release][full]
=> "23"
```



For a more complete reference about [Facts](#) lookup in the Puppet Debugger homepage.

FactorDB

FACTERDB

FactorDB and fact sets

The puppet-debugger internally leverages the [facterdb](#) gem to load pre-cached facts into the debugger session. This is partly why it starts up so fast. You can tell the debugger to use a different set of facts from a different kernel, OS version, facter version or any custom combination that facterdb supports. This is all controlled via the facterdb-filter.

At some point you may need to override the debugger's default fact set with something of your own choosing. You can do this by setting one of the environment variables below.

- `DEBUGGER_FACTERDB_FILTER` Sets the entire filter
- `DEBUGGER_FACTER_OS_VERSION` Sets the OS version (only works with RHEL clones and Fedora)
- `DEBUGGER_FACTER_OS_NAME` Sets the OS name (only works with RHEL clones and Fedora)
- `DEBUGGER_FACTER_VERSION` Sets the facter version to use in the debugger session (uses facter 3.1 for puppet 4.4+ and facter 2.4 for 3.8-4.4)

By default the entire filter looks something like `operatingsystem=Fedora` and `operatingsystemrelease=23` and `architecture=x86_64` and `facterversion=/^3\\.1/` depending on the puppet version currently running.

Below are some examples of loading up different fact sets for various operating systems and then running a lookup for each example fact set.

```
puppet debugger --facterdb-filter 'facterversion=/^3.6./ and operatingsystem=windows' --test -e '$os[family]'
puppet debugger --facterdb-filter 'facterversion=/^3.6./ and osfamily=RedHat' --test -e '$os[family]'
puppet debugger --facterdb-filter 'facterversion=/^3.6./ and osfamily=Darwin' --test -e '$os[family]'
```

You can force the debugger to use real facts from the current node too:

```
puppet debugger --no-facterdb --test -e '$os[family]'
```

You can display the current facterdb filter by running `facterdb_filter` from the debugger session.

Why do facter versions matter? While facter and puppet work independently just fine, newer forge modules utilize certain data structures found only in newer versions of facter. So its really up to the puppet code you use. With facter 3, almost every fact is now a data structure. So if you are having issues with facts not existing in the debugger you may need to change the facter version `DEBUGGER_FACTER_VERSION` to something different.

Custom FacterDB facts

As of FacterDB 0.4.0 we can now supply custom [external fact sets](#) to facterDB.. This is a huge deal because you can take the facts from any system and play around with the fact data in the debugger. This makes the debugger appear as it was run on that system. Moreover, you can create shareable fact sets for your entire team. To use an external fact set you first need to grab the facts from the system you wish to “mock”. This can be done with puppet facts or even puppetdb queries and then create a file with the values hash.

Take the values from the values hash only and place inside a new file.

```
{
  "name": "macbook-pro-10.domain",
  "values": {
    "custom_datacenter_fact": "iceland",
    "puppetversion": "5.3.2",
    "architecture": "x86_64",
    "kernel": "Darwin",
    "domain": "domain"
  }
}
```

This can be easily done if you have jq installed.

```
puppet facts | jq '.values' > /tmp/custom_facts/datacenter_a/2.4/os_x.facts
```

Once you have created a directory with one or more fact files you just need to set an environment variable to tell facterDB about the custom fact sets.

```
export FACTERDB_SEARCH_PATHS="/tmp/custom_facts/"
```

Now everytime you start the puppet debugger, facterDB will use these custom fact files as part of the facterDB database. However, you will still need to provide a search filter that references your custom fact because there are over a thousand facterDB fact sets.

```
puppet debugger --facterdb-filter 'custom_datacenter_fact=iceland'
```

Alternatively you can bypass the internal facterDB database by setting `export FACTERDB_SKIP_DEFAULTDB=true` and force your custom facts to be the only facts facterDB uses.



For a more complete reference about [FacterDB](#) lookup in the Puppet Debugger homepage.

Help

HELP

The help command displays the current runtime information for this debugger session. Because you might have different rubies and puppet runtimes, it is helpful to have this info.

```
I>>> help
Ruby Version: 2.5.1
Puppet Version: 6.4.0
Puppet Debugger Version: 0.12.3
Created by: NWOps <corey@nwops.io>
Type "commands" for a list of debugger commands
or "help" to show the help screen.
```

See the `commands` command for help with all the commands.



For a more complete reference about [Help](#) lookup in the Puppet Debugger homepage.

Exit

EXIT

The exit command will exit the current debugger session.

If you are using the `debug : break()` function and are multiple debugger sessions deep due to enumeration or looping constructs the `exit` command will exit the current loop and allow compilation to proceed. Additionally, `exit` commands may be required to get to the top level of the session.



For a more complete reference about [Exit](#) lookup in the Puppet Debugger homepage.

Types

TYPES

The types command is used to list all the discoverable native resource types you can use in your puppet code.

```
11:>> types
[
  [ 0] "stage",
  [ 1] "whit",
  [ 2] "schedule",
  [ 3] "tidy",
  [ 4] "resources",
  [ 5] "group",
  [ 6] "file",
  [ 7] "notify",
  [ 8] "filebucket",
  [ 9] "exec",
  [10] "package",
  [11] "service",
  [12] "user",
  [13] "file_line",
  [14] "anchor",
  [15] "zpool",
  [16] "zfs",
  [17] "selboolean",
  [18] "selmodule",
  [19] "scheduled_task",
  [20] "sshkey",
  [21] "ssh_authorized_key",
  [22] "yumrepo",
  [23] "mount",
  [24] "augeas",
  [25] "zone",
  [26] "host",
  [27] "cron"
]
```



Due to technical limitations there may be items not in the list that are present in your modulepath.



For a more complete reference about [Types](#) lookup in the Puppet Debugger homepage.

Functions

FUNCTIONS

Puppet Debugger was designed to help you determine what a function returns in real time as well as what functions are available from each module.

The debugger will search your module path for functions and present you with a nice list of functions for you to explorer and execute.

The debugger outputs a table that displays the name and the module from which it was found. Functions found in Puppet core are labeled with the version of Puppet currently running.

To list the functions just use the command functions

```
4:>> functions
FULL_NAME | MOD_NAME
-----|-----
abs       | puppet-6.4.0
alert     | puppet-6.4.0
all       | puppet-6.4.0
annotate  | puppet-6.4.0
any       | puppet-6.4.0
any2array | stdlib
any2bool  | stdlib
assert_private | stdlib
assert_type | puppet-6.4.0
base64    | stdlib
basename  | stdlib
binary_file | puppet-6.4.0
bool2num  | stdlib
bool2str  | stdlib
```



For a more complete reference about [Functions](#) lookup in the Puppet Debugger homepage.

Commands

COMMANDS

If you forget which commands are available, use the commands command.


```
l:>> commands
Context
  environment      Show the current environment name
  reset            Reset the debugger to a clean state.
  whereami         Show code surrounding the current context.

Editing
  play            Playback a file or URL as input.

Environment
  datatypes       List all the datatypes available in the environment.
  functions       List all the functions available in the environment.
  types          List all the types available in the environment.

Help
  commands       List all available commands, aka. this screen
  exit           Quit Puppet Debugger.
  help          Show the help screen with version information.

Node
  classification  Show the classification details of the node.
  facterdb_filter Set the facterdb filter
  facts         List all the facts associated with the node.

Scope
  classes       List all the classes current in the catalog.
  krt           List all the known resource types.
  resources     List all the resources current in the catalog.
  set           Set the a puppet debugger config
  vars         List all the variables in the current scopes.

Tools
  benchmark     Benchmark your Puppet code.
```

If you forgot which command shows the commands, use the `help` command.



For a more complete reference about [Commands](#) lookup in the Puppet Debugger homepage.

Datatypes

DATATYPES

The `datatypes` command will return all the datatypes found in the current module path. This can be handy if you need to find a datatype to use or know which module it came from.

```
5:>> datatypes
[
  [ 0] "Error",
  [ 1] "Stdlib::Compat::Absolute_path",
  [ 2] "Stdlib::Compat::Array",
  [ 3] "Stdlib::Compat::Bool",
  [ 4] "Stdlib::Compat::Float",
  [ 5] "Stdlib::Compat::Hash",
  [ 6] "Stdlib::Compat::Integer",
  [ 7] "Stdlib::Compat::Ip_address",
  [ 8] "Stdlib::Compat::Ipv4",
  [ 9] "Stdlib::Compat::Ipv6",
  [10] "Stdlib::Compat::Numeric",
  [11] "Stdlib::Compat::String",
  [12] "Stdlib::Ensure::Service",
  [13] "Stdlib::Filemode",
  [14] "Stdlib::Filesource",
  [15] "Stdlib::Fqdn",
  [16] "Stdlib::HTTPUrl"
```



For a more complete reference about [Datatypes](#) lookup in the Puppet Debugger homepage.

Classification

CLASSIFICATION

Remote nodes and classification

The puppet-debugger has the ability to pull in remote node information. Instead of mocking facts you can pull in real information! Get real facts and real node classification from the puppet master and then play with the info in the debugger session. This is made possible by using the puppet node indirector interface. What this ultimately means is debugging a node in production without needing access to the node.

ie. `puppet node find hostname --terminus rest --render-as yaml`

In order to use this feature you will need to setup a few things.

Setup

1. Allow node rest calls from your workstation on the puppet master(s) See <https://github.com/nwops/puppet-debugger/issues/17> for more info on allowing this rule.
2. Ensure you have the same code that your puppet master does. Since this pulls in classification you will need to have all the puppet modules defined in your `environmentpath` or `basemodulepath`. The simple thing to do is to point your `basemodulepath` to your fixtures directory if using `rspec-puppet` or some other high level modules directory. For PE users you will want to ensure you have all the pe specific modules in your development environment. Not doing this step will result in class not found errors.
3. Set the server config in your local `puppet.conf`. (on your development machine)

- a. Find your puppet config file `puppet config print config`
 - b. edit the config file with an editor
 - c. add `server = <hostname>` to your main block
 - d. you can also start the debugger with the `puppet debugger --server <hostname>` config set.
4. If using hiera you also need to set the path to your hiera config file since the puppet-debugger will be compiling puppet code which could in turn call hiera functions. Additionally, make sure your hiera config points to the correct data location.

```
[main]
server = pe-puppet.localdomain
basemodulepath = /Users/cosman/github/puppet-debugger-demo/modules:/Users/cosman/Downloads/pe_modules
hiera_config = /Users/cosman/github/puppet-debugger-demo/hieradata/hiera.yaml
```



If using hiera-eyaml or similar plugin that requires keys normally stored on the master you would also need those keys present on your system.



For a more complete reference about [Classification](#) lookup in the Puppet Debugger homepage.

Play

PLAY

Playback support

Playing back files or urls simply loads the content into the debugger session. This means if you want to start a debugger session from an existing file or url you can play the content back in the debugger with the play command. You can also playback a file that contains puppet code and debugger commands.

play <https://gist.githubusercontent.com/logicminds/f9b1ac65a3a440d562b0/raw>

or

puppet debugger -p <https://gist.githubusercontent.com/logicminds/f9b1ac65a3a440d562b0/raw>

or just use STDIN puppet debugger <some_file> or echo "abs(-3)" | puppet debugger

Web based playback support

If using the [web based debugger](#) you can playback a shared url which would start a debugger session and then load the content from the url or parameter.

Example: <https://demo.puppet-debugger.com/play?url=https://gist.github.com/logicminds/64f0fe9f64339f18f097a9f42acd6276>

or for single commands

<https://demo.puppet-debugger.com/play?content=vars>

Please note the web based debugger only contains a minimal amount of puppet modules. So its likely that your code may not work if using third party modules. This may change in the future though.

You may have seen me (nwops) paste code snippets in the Puppet community Slack channel that references a url. You can do this yourself with a simple gist or url hosted on the internet via the http play resource and url parameter.

<https://demo.puppet-debugger.com/play?url=https://gist.github.com/logicminds/4f6bcfd723c92aad1f01f6a800319fa4>

Or inline a small code snippet with the content parameter.

[https://demo.puppet-debugger.com/play?content=md5\('dafsdffdsfasdfds'\)](https://demo.puppet-debugger.com/play?content=md5('dafsdffdsfasdfds'))

The demo site is a web application that runs a local instance of the debugger per session.

So create a link, test it and share it.



For a more complete reference about [Play](#) lookup in the Puppet Debugger homepage.

Reset

RESET

The reset command throws away the current scope and allows you to start all over again without exiting the session. This is useful when redefining variables or other items.

If you just want to clear the screen you can use the `ctrl-l` key combo. (Works in any command line app)

✔ For a more complete reference about [Reset](#) lookup in the Puppet Debugger homepage.

Environment

ENVIRONMENT

The environment command just prints out the currently set puppet environment.

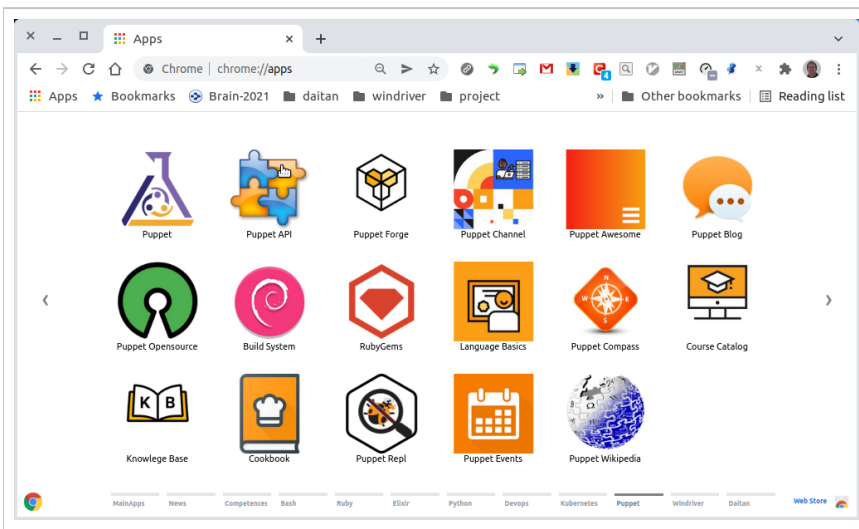
```
11:>> environment
Puppet Environment: production
```

✔ For a more complete reference about [Environment](#) lookup in the Puppet Debugger homepage.

Puppet Extensions

Chrome

[Chrome](#) and [VSCode](#) can help you stay productive and get more out of your browser and code editor.



✔ Chrome Apps

[Puppet](#) - Chrome apps for Puppet configuration management.

For further info about Chrome Apps check [Create and publish custom Chrome apps & extensions](#).

✔ VSCode Extensions

[puppet.puppet-vscode](#) - Puppet Visual Studio Code Extension.

For further extensions check [VSCode marketplace](#).

References



References

1. [Puppet in Wikipedia](#)
 2. [Docker Installation](#)
 3. [Puppet Language Basics](#)
 4. [Puppet Debugger](#)
 5. [Using the Puppet Debugger for Lightweight Exploration -Corey Osman](#)
 6. [Puppet VS Code Tips and tricks](#)
-